Prédire les cas de COVID-19 à sept jours

Une chaîne MLOps complète, des données publiques mondiales jusqu'à une API d'inférence conteneurisée : modèle versionné, promu par comparaison, surveillé en production et réentraîné automatiquement.

RÉALISÉ PAR

Luvengani Dass
Saadia Szapiro

DONNÉES

Our World in Data
France · 2020 → 2023

MODÈLE

Random Forest
11 variables · horizon J+7

LIVRABLE

8 services Docker
API · DVC · MLflow · Airflow · Prometheus

Anticiper une semaine à l’avance, avec ce que l’on sait aujourd’hui

- **La question posée.** À partir de l’historique connu le jour J, estimer le nombre de cas qui sera publié le jour J+7.
- **Pourquoi sept jours.** C’est le délai utile : une semaine suffit pour ajuster une capacité de dépistage, un planning hospitalier ou une campagne de rappel, et reste assez court pour que le passé récent porte de l’information.
- **La contrainte de rigueur.** Aucune variable postérieure au jour J ne doit entrer dans le modèle, sous peine de fuite temporelle — une erreur qui produit d’excellents scores et un modèle inutilisable.

FORMULATION

Régression supervisée sur série temporelle. Une ligne = un jour. La cible est la valeur observée sept jours plus tard.

CE QUE CE N’EST PAS

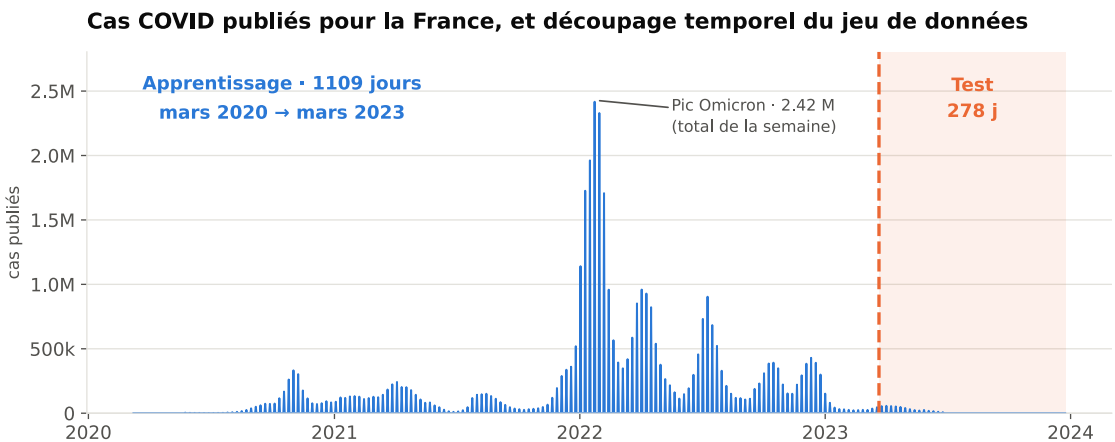
Pas un modèle épidémiologique compartimental (SIR, SEIR). Pas un modèle causal : il apprend des régularités statistiques, il n’explique pas la transmission.

De 429 435 lignes mondiales à 1 387 jours français

Six colonnes retenues sur l'export public d'OWID, filtrées sur la France, triées chronologiquement, puis bornées à la fenêtre d'étude.

`new_cases`	cas publiés — la cible
`new_deaths`	décès publiés
`people_vaccinated`	personnes vaccinées, cumul
`stringency_index`	sévérité des restrictions, 0–100

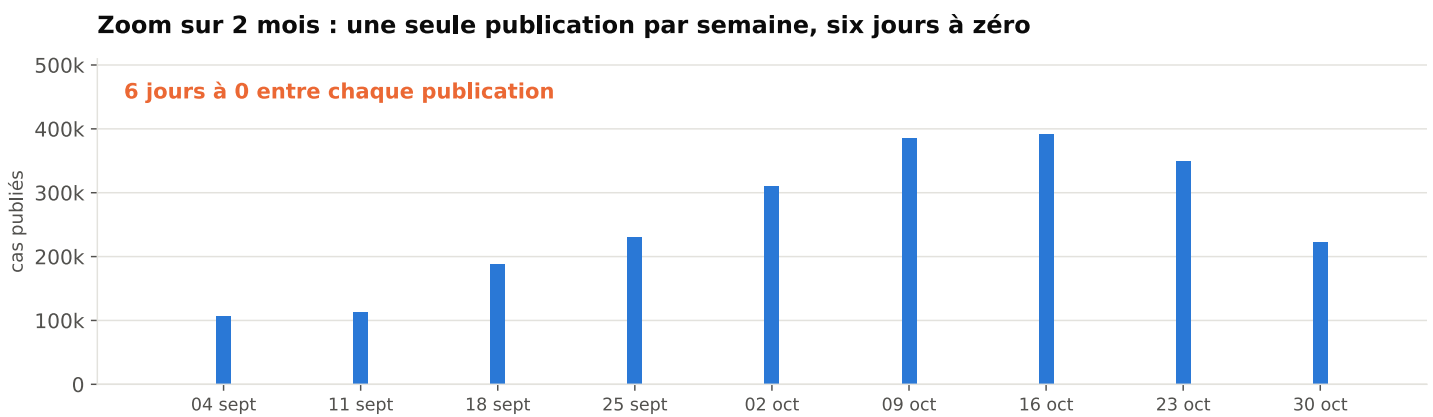
Valeurs manquantes comblées par 0, cas négatifs ramenés à 0 (corrections rétroactives des bulletins).



Fenêtre d'étude : 8 mars 2020 → 24 décembre 2023. Le découpage est strictement chronologique — le modèle n'apprend que sur le passé.

88 % des journées valent zéro

La France ne publie ses chiffres qu'une fois par semaine. Les six autres jours sont absents du fichier source — et le prétraitement les remplace par 0. La série n'est donc pas quotidienne : c'est un **total hebdomadaire posé sur un seul jour**, entouré de six zéros.



1 224 / 1 387
JOURS À ZÉRO, JEU COMPLET

265 / 278
JOURS À ZÉRO, JEU DE TEST

13
VRAIES PUBLICATIONS À PRÉDIRE

Onze variables construites par une fenêtre glissante

ENTRÉES — 7 JOURS CONNUS



CONTEXTE DU JOUR J

new_deaths | people_vaccinated | stringency_index

Sept décalages, une moyenne mobile, trois variables de contexte — onze entrées, aucune information postérieure au jour J.

LE DÉCALAGE DE LA CIBLE

```
df["target"] = df["new_cases"].shift(-7)
```

Chaque ligne apprend à relier son passé immédiat à ce qui arrivera une semaine plus tard.

APRÈS NETTOYAGE

1 387 lignes exploitables sur 1 401 jours de calendrier. Quatorze journées, en tête et en queue de série, sont écartées : les décalages ou la cible y sont incalculables.

Une forêt aléatoire, entraînée dans l'ordre du temps

CHOIX	VALEUR
`RandomForestRegressor`	scikit-learn
`n_estimators`	100 arbres
`random_state`	42 — résultats reproductibles
`profondeur`	non bornée (défaut)

Un ensemble d'arbres capte des seuils et des interactions non linéaires sans exiger de normalisation ni d'hypothèse de distribution — un point de départ solide sur onze variables hétérogènes.

LE POINT CRITIQUE : PAS DE MÉLANGE

Le découpage est **chronologique**, jamais aléatoire. Mélanger les lignes ferait apprendre au modèle des jours postérieurs à ceux qu'il doit prédire : les scores s'envoleraient et le modèle ne vaudrait rien en production.

1 109 jours

APPRENTISSAGE

mars 2020 → mars 2023

278 jours

TEST

mars → déc. 2023

Le modèle bat nettement les références naïves

943 cas

MAE — ERREUR ABSOLUE MOYENNE

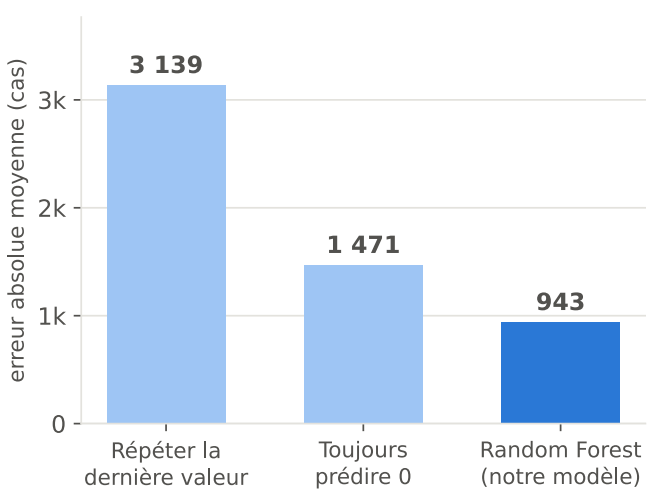
4 262 cas

RMSE — ERREUR QUADRATIQUE

0,68

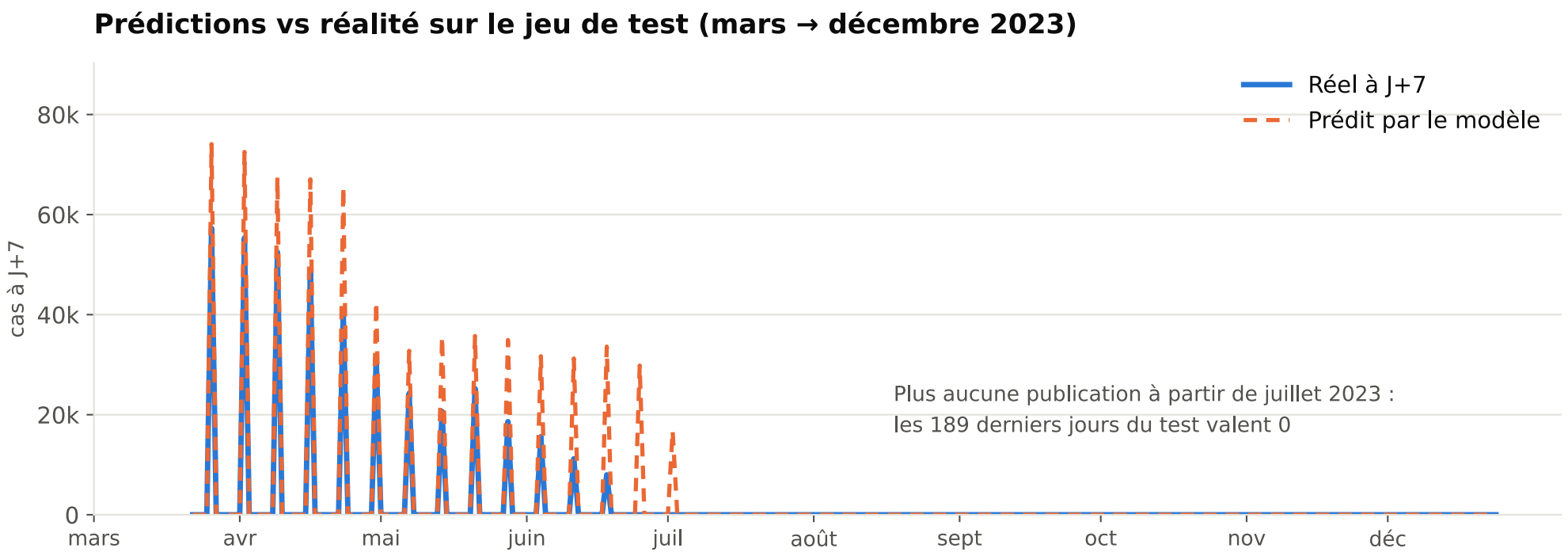
R² — VARIANCE EXPLIQUÉE

MAE : le modèle face à deux références simples



- **−70 % d'erreur** face à la référence « répéter la dernière valeur connue » (3 139 cas de MAE).
- **−36 % d'erreur** face à la référence « toujours prédire zéro » (1 471 cas) — la barre la plus exigeante, vu la structure des données.
- Le **rapport RMSE / MAE de 4,5** est le vrai signal à lire : il dit que l'erreur n'est pas répartie uniformément. La diapositive 09 montre où elle se concentre.

Le modèle a compris le rythme, pas encore l'amplitude

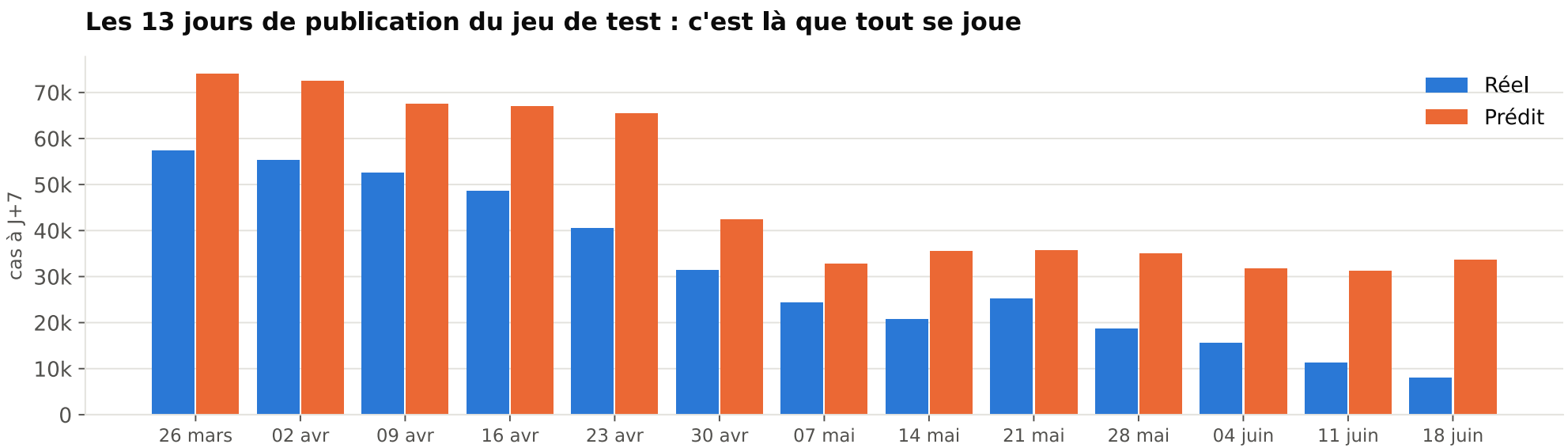


Le rythme est juste. Les pics prédits tombent exactement les jours de publication : la périodicité hebdomadaire est parfaitement restituée.

L'amplitude est surestimée. Sur les treize publications, la prédiction dépasse le réel à chaque fois — le modèle reste ancré sur les niveaux, plus élevés, vus à l'entraînement.

Puis plus rien. La France cesse toute publication après le 18 juin 2023 : les 189 derniers jours du test sont des zéros que le modèle prédit sans effort.

Toute l'erreur se concentre sur treize journées



176 cas

MAE SUR LES 265 JOURS SANS PUBLICATION

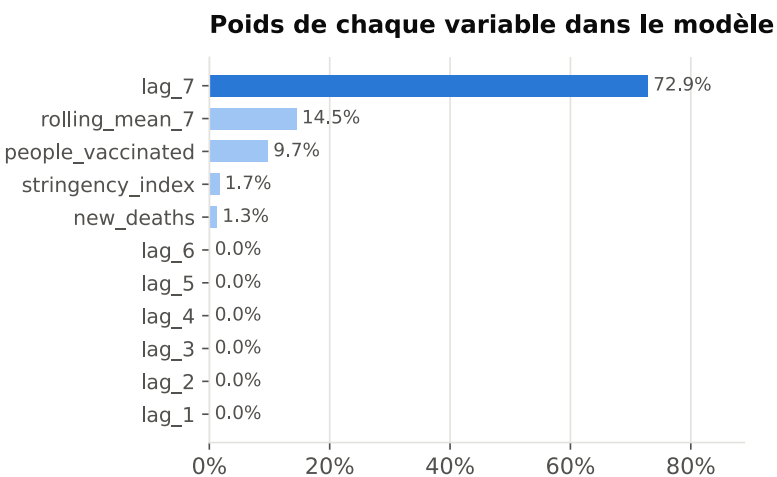
16 576 cas

MAE SUR LES 13 JOURS DE PUBLICATION

77 %

PART DE L'ERREUR QUADRATIQUE DUE À CES 13 JOURS

Voilà pourquoi la MAE est flatteuse : moyennée sur 278 jours dont 265 sont triviaux, elle dilue l'erreur réelle d'un facteur dix-sept.

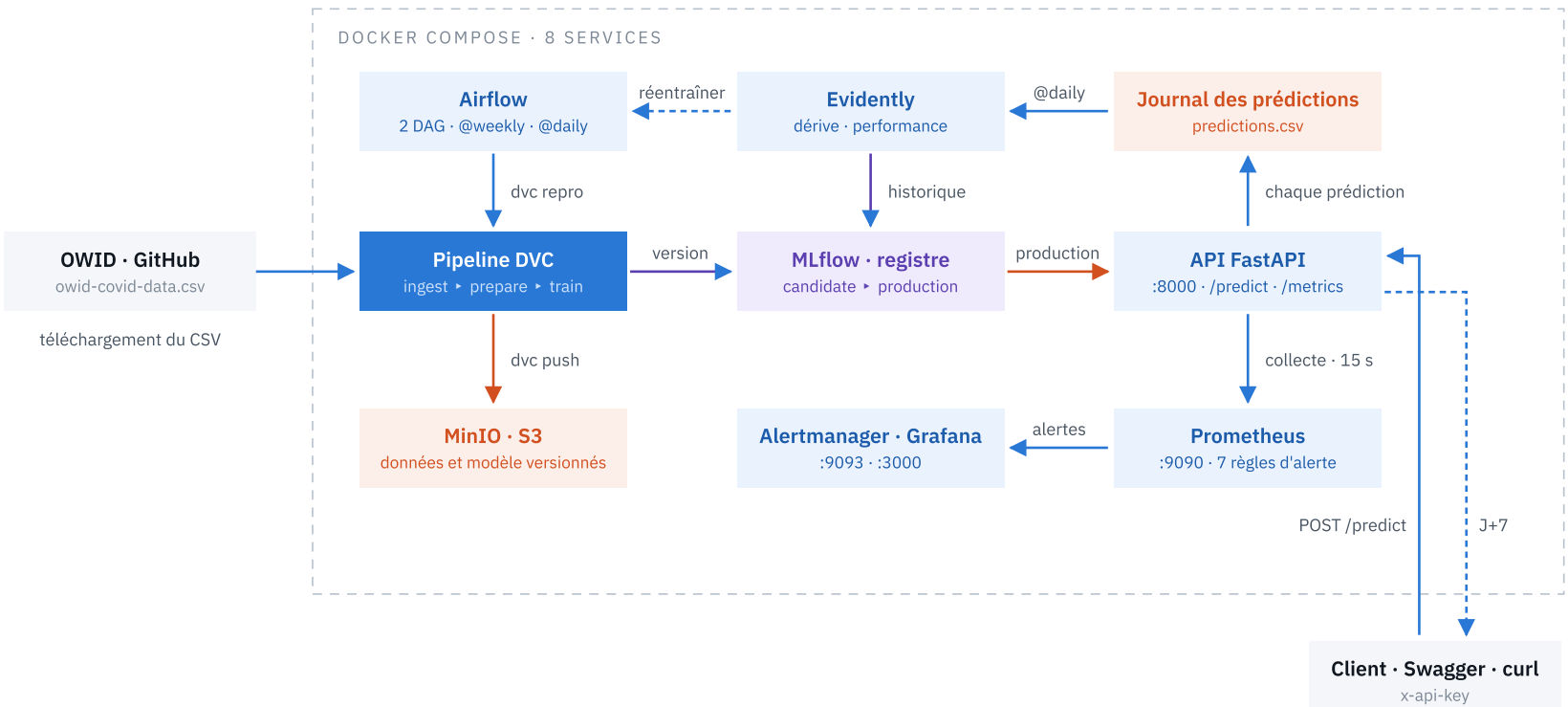


Le modèle a redécouvert seul la cadence hebdomadaire

- **lag_7 pèse 72,9 %** à lui seul : la dernière publication connue, exactement sept jours plus tôt, est de loin le meilleur prédicteur.
- **lag_1 à lag_6 pèsent 0,0 %** — pas « presque zéro », zéro exact. Ces six variables valent 0 six fois sur sept : elles ne portent strictement aucune information.
- La **vaccination cumulée (9,7 %)** joue en réalité le rôle d'un indicateur de calendrier : elle croît de façon monotone et situe la date dans l'épidémie.

Ce graphique est le meilleur diagnostic du projet : il confirme, depuis l'intérieur du modèle, le défaut de structure identifié à la diapositive 04.

Huit services, trois boucles : entraîner, servir, surveiller



Trois exigences, trois réponses : le versioning des données et des modèles (DVC, MinIO, MLflow), le registre et son cycle de vie (MLflow, Airflow), le monitoring en production (Prometheus, Alertmanager, Grafana, Evidently). Les diapositives 13 à 15 les détaillent.

Cinq points d'entrée, une clé, une réponse

ENDPOINT	RÔLE
`GET /health`	modèle chargé, version, origine
`POST /predict`	prévision — protégé par clé
`POST /model/reload`	recharge la version en production — protégé
`GET /metrics`	métriques pour Prometheus
`GET /docs`	documentation Swagger générée

- **Validation Pydantic** des onze champs : un type erroné est rejeté avant d'atteindre le modèle.
- **Clé API** en en-tête `x-api-key` ; absente ou incorrecte, la requête reçoit un `401`.
- **Modèle lu dans le registre** : la version qui porte l'alias `production` ; si MLflow est injoignable, repli sur le fichier local.

REQUÊTE

```
POST /predict
x-api-key: mlops-covid-secret-key

{
  "lag_1": 1200,  … "lag_7": 1500,
  "rolling_mean_7": 1300,
  "new_deaths": 25,
  "people_vaccinated": 40000000,
  "stringency_index": 45.5,
  "date": "2023-03-01"
}
```

RÉPONSE

```
200 OK

{ "prediction_new_cases_h_plus_7": 5300.66,
  "model_version": "4" }
```

À partir d'un modèle, retrouver ses données et son code

DVC — LES DONNÉES ET LE MODÈLE

- Un pipeline en trois étapes : `ingest`, `prepare`, `train`.
- `dvc.lock` garde l'**empreinte** de chaque fichier, de chaque script et des paramètres : un commit git désigne une version exacte du code, des données et du modèle.
- `dvc repro` ne rejoue que les étapes dont une entrée a changé ; `dvc push` envoie les fichiers dans **MinIO**, un stockage compatible S3.

Retour arrière : `git checkout` puis `dvc checkout`.

MLFLOW — LE LIEN ENTRE MODÈLE ET DONNÉES

- Chaque entraînement enregistre ses **9 paramètres** et ses **2 métriques**, MAE et RMSE.
- Il enregistre aussi le jeu de données utilisé, son empreinte DVC et le commit git.

DERNIER ENTRAÎNEMENT

`Version du modèle`	covid_rf_model v4
`Empreinte des données`	d7bac4c8f0bc…
`Run MLflow`	b0eb5ae8db13…

Un modèle n'entre en production que s'il fait au moins aussi bien

MLFLOW — UN CYCLE DE VIE PORTÉ PAR DEUX ALIAS

- `candidate` : la dernière version entraînée, à valider.
- `production` : la version servie par l'API.
- Le candidat et la production sont évalués **sur le même jeu de test** ; le candidat ne passe que si sa MAE est inférieure ou égale. Chaque version porte son verdict : promu, rejeté, remplacé.

Les alias remplacent les étapes Staging / Production, que MLflow a dépréciées. Retour arrière : déplacer l'alias `production`, puis appeler `POST /model/reload`.

AIRFLOW — LE DAG RETRAIN_COVID_MODEL

- Six tâches : `ingest_data`, `prepare_data`, `train_model`, `promote_model`, `version_artifacts`, `reload_api`.
- Planifié `@weekly`, à la cadence de publication des données ; déclenchable à la main.
- `reload_api` fait charger la nouvelle production à l'API, sans redémarrage.

DERNIÈRE PROMOTION

Version en production	v4
MAE du dernier candidat	942,72 cas
Décision	le candidat est déjà en production

Surveiller le service, surveiller le modèle, réagir seul

PROMETHEUS — LE SERVICE

- L'API expose `/metrics`, collecté toutes les **15 secondes** : requêtes, temps de réponse, prédictions par version du modèle.
- **7 règles d'alerte** : API indisponible, aucun modèle chargé, taux d'erreur, latence, dérive, performance dégradée, monitoring en retard.
- **Alertmanager** regroupe les alertes et les achemine ; **Grafana** les affiche.

Le canal de notification (mail, Slack) reste à brancher.

EVIDENTLY — LE MODÈLE

- Chaque prédiction est journalisée avec ses entrées.
- **Dérive** : alerte si au moins la moitié des variables ont changé de distribution par rapport à l'entraînement.
- **Performance** : les prévisions datées sont rapprochées de la valeur observée à J+7 ; alerte au-delà de 1,5 fois l'erreur relative de référence.
- Le DAG `monitor_covid_model` tourne `@daily` et **relance l'entraînement** si nécessaire.

DERNIER CONTRÔLE · 2026-10-12

Prédictions analysées	478
Variables en dérive	9 sur 11
Réentraînement recommandé	oui

La chaîne en fonctionnement, du démarrage au réentraînement

01 **`docker compose up -d`**
Les huit conteneurs démarrent ensemble : API, MLflow, MinIO, Airflow, Prometheus, Alertmanager et Grafana.

02 **Swagger sur `:8000/docs`**
L'API se documente seule : les onze champs attendus, leurs types et un exemple pré-rempli.

03 **`GET /health`**
Le modèle répond avec sa propre carte d'identité : version, origine (le registre), pays, horizon, MAE et RMSE.

04 **`POST /predict` sans clé → 401**
La protection se vérifie en une requête : sans en-tête `x-api-key`, rien ne passe.

05 **`POST /predict` avec la clé → 200**
Sur la charge utile d'exemple, le modèle renvoie **5 300,66 cas** à J+7.

06 **MLflow sur `:5000`**
Les runs côte à côte, et dans l'onglet Models les versions avec leurs alias `candidate` et `production`.

07 **Airflow sur `:8080` → Trigger DAG**
`retrain_covid_model` se déclenche à la main ; ses six tâches passent au vert, de l'entraînement à la promotion.

08 **Grafana sur `:3000`**
Après un trafic de démonstration et le DAG `monitor_covid_model`, la dérive apparaît sur le tableau de bord et l'alerte dans Alertmanager.

PLAN DE REPLI

Si un service tarde à démarrer, le récit se déroule à l'identique dans l'application Streamlit, dont le simulateur appelle directement le modèle `models/covid_rf_model.joblib`, sans Docker.

Ce qui est vérifié, et ce qui ne l'est pas encore

8 / 8

TESTS PYTEST RÉUSSIS

0

ÉCHEC

TEST	ATTENDU
`/health`	200 · modèle chargé
`/predict sans clé`	401
`/predict avec clé, avec ou sans date`	200 · prévision
`/metrics · /model/reload`	200
`Construction des variables`	décalages, cible, lignes incomplètes

Les tests de l'API s'exécutent contre l'application réelle via le `TestClient` de FastAPI. Deux tests de plus, sur les règles de dérive et de performance, s'exécutent dans l'image Airflow, où Evidently est installé.

CONTENEURISATION

Trois images construites (API, MLflow, Airflow), cinq images publiques et huit services orchestrés par un seul `docker compose up`. Les volumes assurent la persistance : modèle, base MLflow, métadonnées Airflow, stockage MinIO.

ANGLES MORTS ASSUMÉS

- Aucun test sur le téléchargement ni sur l'entraînement lui-même.
- Pas de test d'intégration bout en bout sur la chaîne de réentraînement, ni d'intégration continue.

Six limites que nous assumons

01 **La cadence hebdomadaire n'est pas traitée.** Travailler sur une série « quotidienne » composée à 88 % de zéros rend les métriques globales flatteuses et peu informatives.

02 **Six variables sur onze sont inertes.** lag_1 à lag_6 ne portent aucune information : le modèle en supporte le coût sans aucun bénéfice.

03 **Surestimation systématique en décrue.** Sur les treize publications testées, l'erreur relative moyenne atteint 82 %.

04 **Tout tourne sur une seule machine.** MinIO, MLflow et Airflow partagent le même poste : ni sauvegarde distante, ni reprise si la machine tombe.

05 **Le secret est en clair.** La clé API figure dans `docker-compose.yml` et dans le dépôt.

06 **Les alertes n'ont pas de destinataire.** Elles sont déclenchées, regroupées et routées, mais aucun canal (mail, Slack) n'est branché.

Ces limites ne sont pas des oublis : elles ont été identifiées en analysant les résultats, et c'est cette analyse qui a produit les diapositives 04, 09 et 10.

La chaîne tient ; c'est le signal qu'il faut maintenant redresser

SUR LA MODÉLISATION

- **Rééchantillonner en pas hebdomadaire.** Une série de 201 points denses remplacerait 1 387 points dont 88 % de zéros — les décalages redeviendraient tous porteurs d'information.
- **Évaluer sur les seuls jours de publication**, et publier systématiquement l'écart à une référence naïve.
- **Valider par TimeSeriesSplit** plutôt que par un unique découpage, et comparer la forêt à une régression régularisée et à un gradient boosting.

SUR L'INDUSTRIALISATION

- **Brancher un canal de notification** sur Alertmanager, mail ou Slack.
- **Héberger le stockage et le registre** : un S3 et une base MLflow hors du poste, sauvegardés ; pour DVC, seule l'adresse change.
- **Sortir les secrets du dépôt**, dans un coffre ou des variables injectées au déploiement.
- **Brancher les tests sur une intégration continue** et couvrir la chaîne de réentraînement de bout en bout.

L'infrastructure MLOps est en place et fonctionne de bout en bout. Le travail restant porte désormais sur la donnée et sur l'évaluation — c'est-à-dire sur ce qui fait la valeur d'une prévision.